

BoostViT: Booth-Serial Skipping and Tunable Scaling for Vision Transformers

Shiqi Zhao¹, Member, IEEE, Sibowang², Zhengjie Wang, Chaoming Fang³,
Fengshi Tian⁴, Graduate Student Member, IEEE, Jinbo Chen⁵, Student Member, IEEE, Changzeng Fu⁶,
Jie Yang⁷, Member, IEEE, and Mohamad Sawan⁸, Life Fellow, IEEE

Abstract—Vision Transformers (ViTs) have emerged as a dominant architecture in computer vision (CV), surpassing conventional neural network counterparts across diverse visual tasks. Despite their exceptional performance, ViTs incur substantial computational overhead characterized by high memory footprint, long inference latency, and elevated energy consumption. Current acceleration strategies for ViTs primarily focus on pruning operations or leveraging the inherent sparsity, requiring complex address control logic or position encoding. Alternatively, some software-based approaches attempt to pre-compute and separate dense and sparse matrix position encoding, while hardware solutions typically spend additional time and resources to obtain position encoding, decomposing matrix multiplications into structured forms. Through an analysis of ViTs' parameters, we found that approximately 91.03% of the most significant bits (MSBs) are either 111s or 000s, and nearly 45% of 3 adjacent bits are identical. To leverage this characteristic of ViTs, we propose the Booth-Serial Skipping algorithm, which transforms the computation of consecutive 111 or 000 sequences into skip steps that require no additional computation time. Furthermore, the 4th to 6th bits of ViT weights can undergo aggressive scaling, enhancing the likelihood of Booth-skip operations with minimal impact on accuracy. The key innovation of this paper lies in exploiting the high proportion of naturally consecutive 0s or 1s in 8-bit weights during ViT inference and further expanding the skippable range through the Tunable Scaling strategy. At the hardware level, we develop a specialized accelerator to coordinate the proposed acceleration strategies. The processing element array in the accelerator is optimized for general matrix multiplication, it not only significantly improves

the computation of multi-head self-attention but also enables resource reuse for linear transformations, ultimately optimizing end-to-end inference. Our design achieves $50.3\times$, $21.9\times$, $17.37\times$, $7.47\times$, and $1.49\times$ an average end-to-end speedup on DeiT over CPU (Intel Xeon Gold 6152), EdgeGPU (NVIDIA Jetson Xavier NX), GPU (TITAN Xp), ViTCoD, and ViT-slice, respectively.

Index Terms—Vision transformer, booth-serial skip algorithm, hardware-software co-design.

I. INTRODUCTION

IN RECENT years, Transformer architectures have demonstrated extraordinary performance in natural language processing (NLP) [1], [2]. Thanks to the ingenious design of Multi-Head Self-Attention (MHSA) [3], Transformers can effectively capture the relationships within the context and allocate different weights to each token, enabling them to excel in capturing global information. ViTs [4], [5] apply this Transformer architecture to the field of computer vision (CV) tasks. Given that there is a strong correlation between split image patches from one entirety, similar to the way words relate in a sentence, ViTs perform exceptionally well in tasks such as object detection [6], semantic segmentation [7], and image retrieval [8]. However, despite their outstanding performance, ViTs require multiple stacked blocks with computationally expensive and hardware-unfriendly MHSA and linear transformation operations, which demand significant computational resources and data throughput [9].

In order to optimize Transformers, a series of highly efficient accelerators have been proposed. **Most state-of-the-art works focused on accelerating the natural language process (NLP) Transformers emphasize the use of sample-based dense-dense matrix multiplication (SDDMM) and sparse-dense matrix multiplication (SpMM) to optimize attention calculations, either at the fine-grained or coarse-grained level [12], [13], [14], [15].** For ViTs, however, there are some unique characteristics. As shown in Fig. 1, unlike the vanilla Transformer, ViTs have fixed input sizes and more structured sparsity in their tokenized image patches. This enables ViTs to undergo more aggressive pruning strategies, achieving minimal accuracy degradation even under high sparsity ratios. As a result, a series of accelerators tailored to ViTs have emerged to take full advantage of these new data characteristics.

There are two main directions in accelerator designs for ViTs:

Received 5 March 2025; revised 4 May 2025; accepted 20 May 2025. Date of publication 9 June 2025; date of current version 27 October 2025. This work was supported in part by the STI2030-Major Projects under Grant 2022ZD0208805, in part by the National Natural Science Foundation of China under Grant 62404037, in part by the Hebei Natural Science Foundation under Grant F2024501017, and in part by the Fundamental Research Funds for the Central Universities under Grant Z2024-043. This article was recommended by Associate Editor J. Park. (Corresponding authors: Shiqi Zhao; Fengshi Tian.)

Shiqi Zhao, Sibowang, Zhengjie Wang, and Changzeng Fu are with the School of Computer and Communication Engineering, Northeastern University, Shenyang 110819, China, and also with Hebei Key Laboratory of Marine Perception Network and Data Processing, Northeastern University at Qinhuangdao, Qinhuangdao, Hebei 066004, China (e-mail: zhaoshiqi@qhd.neu.edu.cn).

Chaoming Fang, Jinbo Chen, Jie Yang, and Mohamad Sawan are with the Center of Excellence in Biomedical Research on Advanced Integrated-on-Chips Neurotechnologies (CenBRAIN Neurotech), School of Engineering, Westlake University, Hangzhou 310024, China, and also with the Institute of Advanced Technology, Westlake Institute for Advanced Study, Hangzhou 310024, China.

Fengshi Tian is with the Department of Electronic and Computer Engineering, The Hong Kong University of Science and Technology (HKUST), Hong Kong, SAR, China (e-mail: fengshi.tian@connect.ust.hk).

Digital Object Identifier 10.1109/TCSI.2025.3575328

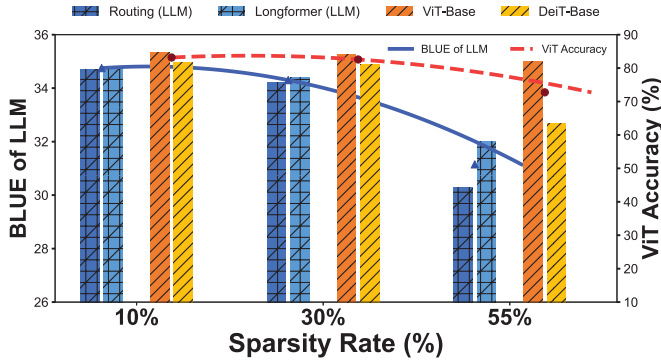
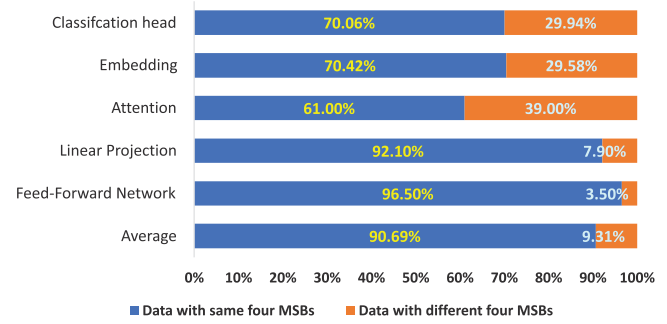


Fig. 1. Comparison of BLEU-sparsity in NLP Transformers and accuracy-sparsity in ViTs. For NLP Transformers, the results are obtained from a machine translation task (IWSLT EN $\rightarrow$ DE) as described in [10]. In the case of ViTs, an information-based pruning technique is employed on DeiT-Base/Small models, with the classification task following [11].

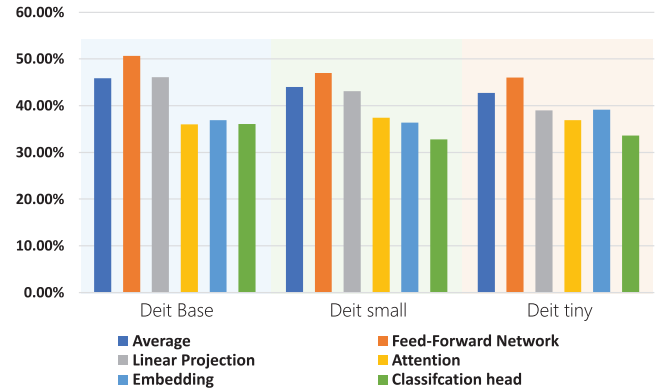
- 1) **Static Sparsity Exploitation**—These accelerators focus on sparse processing for attention, often co-designed with the SpMM and SDDMM engines like SpAtten, Sanger, and ViTCoD [12], [13], [14].
- 2) **Dynamic Runtime Pruning**—These accelerators detect and remove non-contributing parts of the image or process only the important tokens or attention heads like DynamicViT and HeatViT [15], [16], [17], [18].

Static Sparsity Exploitation based approaches typically amplify the natural sparsity of ViT weights (10% to 15%) through thresholds [15], [20], [21], [22], which are often accompanied by mask/index operations that divide one GEMM into SDDMMs and SpMMs. These operations introduce extra computation to generate complex masks which leads to unbalanced workloads between SDDMM and SpMM [23], [24], ultimately reducing resource reuse. Additionally, these optimizations primarily target attention operation, without fully leveraging other characteristics of ViTs, which limits their end-to-end performance. On the other hand, Dynamic Runtime Pruning based accelerators inevitably disrupt the integrity of the image and the computational accuracy of the ViT model, leading to accuracy degradation. Moreover, most of these methods primarily reduce the number of tokens rather than being achieved through deeply coordinated hardware designs [18], [25]. **Therefore, a key challenge we aim to address is how to better leverage the inherent data characteristics of ViTs and propose a hardware and software co-designed framework.**

In this paper, we propose **BoostViT**, an efficient hardware-software co-design framework that avoids the potential drawbacks of existing acceleration methods. We observe that approximately 90.69% of the first four MSBs in ViT weights are the same in Fig. 2, and nearly 45% of the adjacent 3-bits are identical, either all 111s or all 000s, which demonstrates an astonishing compatibility with the hardware-friendly Booth algorithm at the bit level. In the context of the Booth algorithm, these all-1s and all-0s can be treated as effectively skipping computations without any time consumption, which we call Booth-Serial Skipping. Additionally, we propose a tunable scaling strategy for the 4th to 6th LSBs based on a threshold, rather than simply zeroing out all 8-bits, which further



(a)



(b)

Fig. 2. The distribution of successive 111s and 0000s of the MSBs (a) and the same 3 adjacent bits of the entire 8-bit (b) in each part within a ViT [19]. These observations illustrate that the identical MSB distribution of the attention part is exceptionally high, and the same 3 adjacent bits of the 8-bit quantized weight of all linear operations also oppose a remarkable proportion.

minimizes the accuracy drops and enhances the Booth-skip rate. To fully utilize this property, we propose a customized accelerator featuring a reconfigurable pipeline, increasing the efficient throughput and accelerating the computation of unstructured sparse matrices.

Compared with other works that focus on the internal sparsity of MHSA [12], [13], [14], our approach integrates the sparsity operation directly into the PE, without requiring additional precomputed or time- and resource-consuming real-time mask/index matrices. This approach not only reduces resource and timing overhead but also provides an additional bonus in end-to-end operations. This will be discussed in detail in the experimental section.

Our contributions can be summarized as follows:

- **On the algorithm level**, we propose the Booth-Serial Skipping Algorithm, which uses thresholds to scale only the 4th to 6th less significant bits (LSBs) of weights, avoiding the detrimental impact of simple zeroing of 8-bit weights on accuracy as well as increasing Booth-skip rate.
- **On the hardware level**, we exploit the MSB characteristics of ViT and design a reconfigurable Booth-Serial Processing Unit (BSPE), enabling the systolic array to meet the sequential requirements with dynamic Booth-Serial Skipping cycles in each BSPE.

Extensive experiments validate the advantages of our proposed BoostViT framework, achieving end-to-end speedups of $50.3\times$, $21.9\times$, $17.37\times$, $7.47\times$, and $1.49\times$ over general computing platforms on DeiT, including CPUs, EdgeGPUs, GPUs, and prior art ViT accelerators ViTCoD and ViT-slice, respectively.

II. BACKGROUND AND RELATED WORKS

A. Efficient ViTs

1) *MHSA and ViTs*: The Transformer architecture was initially designed for NLP tasks [3], then Dosovitskiy [4] and Carion et al. [6] adapted the Transformer architecture for CV tasks by treating images as input. This adaptation led to significant improvements in recognition performance, surpassing traditional CNN-based implementations [26], [27], [28]. The primary factors contributing to this enhanced performance include the more advanced training techniques and the use of larger-scale datasets. The ViT architecture employs a linear projection for input embedding, followed by several stacked blocks that incorporate Multi-Head Self-Attention (MHSA) and Multi-Layer Perceptron (MLP) layers, concluding with a classification layer. MHSA is the fundamental component of Transformers, consisting of multiple attention heads (denoted as H). Each attention head captures distinct global contextual information by evaluating pairwise correlations between all tokens, as defined below:

$$O_{\text{Attn}} = \text{Concat}(H_1, \dots, H_h) \cdot W^O, \text{ where} \\ H_i = \text{Softmax} \left(\frac{XW_i^Q \cdot (XW_i^K)^T}{\sqrt{d_k}} \right) \cdot XW_i^V \quad (1)$$

where h represents the number of overall attention heads, and the query, key, and value matrices, Q , K , and V (each in $\mathbb{R}^{n \times d}$), are obtained by linearly projecting an input sequence of length n . For each head i , the learnable projection matrices W_i^Q , W_i^K , and W_i^V (each in $\mathbb{R}^{d \times d_k}$) are employed, where $d_k = d/h$ specifies the embedding dimension for each head. The MLP consists of two fully connected linear layers, followed by the Gaussian Error Linear Unit (GELU) activation function. For ViT, the architecture mirrors that of NLP Transformers, but with images split into patches as the input. After embedding the input images, the tokens pass through multiple encoder blocks and ultimately produce classification scores.

2) *Works on ViTs Optimization*: As a trade-off for its impressive performance, the Transformer model requires significant redundant computation, leading to high inference latency and challenging deployment on resource-constrained devices. Consequently, many subsequent works have focused on enhancing the efficiency of ViT inference. DeiT [29] employs knowledge distillation, achieving high accuracy without the need for pre-training. Models such as CrossViT [30], PiT [31], PVT [32], and SwinTransformer [33] explore various structural optimizations for ViTs. For hardware-software co-optimization, works such as CvT [34], MobileViT [35], ViTCoD [14], BSViT [36], and ViT-slice [19] have been proposed. Among them, LeViT [37], CvT [34], MobileViT [35], and ViTCoD [14] focus on optimizing the computationally intensive attention module. Some accelerators also try to

optimize ViT at the bit level. ViT-slice [19] attempts to achieve an end-to-end speedup for the MSBs characteristics, however, it has only realized approximately 25% of its potential. BSViT [36] divides the 8-bit weight into 4-bit MSBs and LSBs and tries to balance the workload between dense and sparse engines, failing to benefit from the native characteristics and achieving limited throughput increase.

B. Overview of the Related Accelerators

1) *NLP Transformer Accelerators*: These accelerators are primarily designed to handle the computationally expensive attention component. A^3 [38] is designed for unstructured sparse matrices, employing a greedy algorithm that accelerates the computation of attention scores by approximating them and searching for the most strongly associated Q vectors with current K vectors. SpAtten [13] utilizes a Top-K selection engine to accelerate inference by disregarding attention heads and tokens with weak associations. Despite its reduced precision, it achieves relatively low sparsity. Sanger [12] generates attention masks by selecting influential QK vectors, and features a clever design for resource reuse. However, unlike NLP, where tokens have distinct relationships, ViT inputs are image patches that have more inherent similarity and correlation, which allows ViT to undergo more aggressive pruning and achieve higher sparsity with acceptable accuracy degradation [25], [39], [40]. For instance, while NLP Transformers aim to preserve precision at a medium sparsity level of around 40%, ViTs can be pruned up to 90% - 95% of their attention maps with fixed sparse patterns across all inputs, resulting in minimal accuracy loss.

2) *Vision Transformer Accelerators*: Existing research primarily focuses on either sparsity or pruning strategies to optimize ViTs [41]. ViTCoD [14] proposes an encoder-decoder engine to prevent data access from becoming the bottleneck under the relatively low memory transfer speed of DDR4. However, it has some limitations: (1) It accelerates only the attention part of ViTs, without providing end-to-end optimizations. (2) Although the encoder-decoder engine may reduce access times, it incurs additional loss in time and accuracy during data restoration. (3) The SDDMM and SpMM engines require extra latency for index processing, resulting in an imbalanced workload and low utilization of the SpMM engine. HeatViT [40] focuses on dynamically pruning tokens to reduce computation. However, (1) It disrupts the integrity of the image, which inevitably affects accuracy, and (2) the hardware is not fully optimized to take advantage of the characteristics of the data. ViT-slice [19] and BSViT [36] attempt to explore the bit-level sparsity of ViT weights, but their designs are overly fine-grained for bit-level operations and do not fully exploit the potential of this characteristic.

III. BOOSTViT: MOTIVATIONS & OVERVIEW

A. Motivations for BoostViT

As discussed in Section II-B, existing approaches for static sparsity exploitation face significant challenges. Specifically, pruning strategies are either overly fine-grained for bit-level operations or too coarse when applied to all-zero scaling.

Furthermore, the unstructured sparsity pattern introduces complex index locating and unbalanced workload, leading to the SDDMM becoming the bottleneck during inference. On the other hand, dynamic runtime pruning methods tend to focus heavily on software optimization without considering hardware co-design. Additionally, most related works fail to exploit the characteristics of the MSBs. As shown in Fig. 2, the distribution of identical 3-bits in the ViT weight is exceptionally high, yet its potential remains underutilized. Inspired by the innovations and limitations of the aforementioned works, we propose BoostViT. Our objectives are: (1) Using Booth-Serial Skipping Algorithm to fully leverage the successive 111s and 000s patterns, thus leveraging their inherent properties while co-designing hardware to optimize performance. (2) Scaling the 4th to 6th LSBs with thresholds instead of the entire 8-bit weight, thereby minimizing accuracy degradation as well as magnifying the natural existing 111s and 000s. (3) Avoiding the unbalanced SDDMM and SpMM workload while enabling resource reuse and improving end-to-end performance.

B. BoostViT Overview

Based on the above motivations, we propose BoostViT, which aims to leverage the data distribution characteristics of ViTs to achieve both improved attention performance and end-to-end speedups. On the software level, we use the Booth-friendly LSB scaling strategy to amplify the natural proportion of the successive 111s and 000s in attention part and attention score early termination to ease the calculation of the attention scores. On the hardware level, we co-design the Booth Skipping Processing Elements, which can perform the Booth Skip operations in systolic array and leverage ViTs' identical 3-bits characteristics. In the following sections, we will provide a detailed description of the proposed BoostViT accelerator.

IV. PROPOSED BOOSTViT ALGORITHM

In this section, we introduce the proposed BoostViT Algorithm. First, by analyzing the weights and activations of 8-bit integers quantized models, we observe that the data distribution within ViTs naturally exists dense sequences of 000s and 111s. Fig. 2 shows the exceptionally high frequency of successive identical first 4-bits MSBs in ViT weights. Specifically, the natural 3-bits identical MSBs (either 000s or 111s) proportion reaches 91.01%, and the entire identical 3-bits proportion is nearly 45%, which means that over 91.01% can benefit from the Radix-4 Booth algorithm, resulting in an overall $1.82\times$ speedup naturally. We also analyzed the K matrices in 4-bit quantized ViTs and also observed high proportion of entirely identical 2-bit segments, which indicates that our architecture can also leverage Radix-2 Booth multiplication to optimize 4-bit ViTs. However, it is important to point out that many existing 4-bit int quantized ViTs still exhibit a noticeable performance gap compared to 8-bit quantized models.

Based on this observation, we introduce three novel perspectives: (1) Proposing Booth-Serial Skipping algorithm which is tailored for the high identical 3-bit proportion. From the perspective of the Radix-4 Booth algorithm, 111s and 000s are

Algorithm 1 The Proposed BoostViT Algorithm

Input: Attention matrices $Q \in \mathbb{R}^{h \times j \times m \times n}$, $K \in \mathbb{R}^{h \times j \times n \times z}$, $threshold_1$, $threshold_2$

Output: Processed attention score matrix

for each encoder do

// 8-bit Quantization

$Q_{int}, K_{int} \leftarrow \text{Quantization}(Q), \text{Quantization}(K)$

// Booth-friendly Transformation

for each element q, k in Q_{int}, K_{int} do

if $k > threshold_1$ **then** $k \leftarrow k \mid 0 \times 1F$

else if $k < threshold_1$ **then** $k \leftarrow k \mid 0xE0$

Obtain modified $K_{modified}$

end

Attention score $\leftarrow \text{Booth}(Q \cdot K_{modified})$

// Attention Score Early Termination

for each element in attention score do

if element $< threshold_2$ **then** element $\leftarrow 0$

else if element $> threshold_2$ **then** Termination

end

Row, Column $\leftarrow$ Address (where Attention score is 0)

end

return encoder result

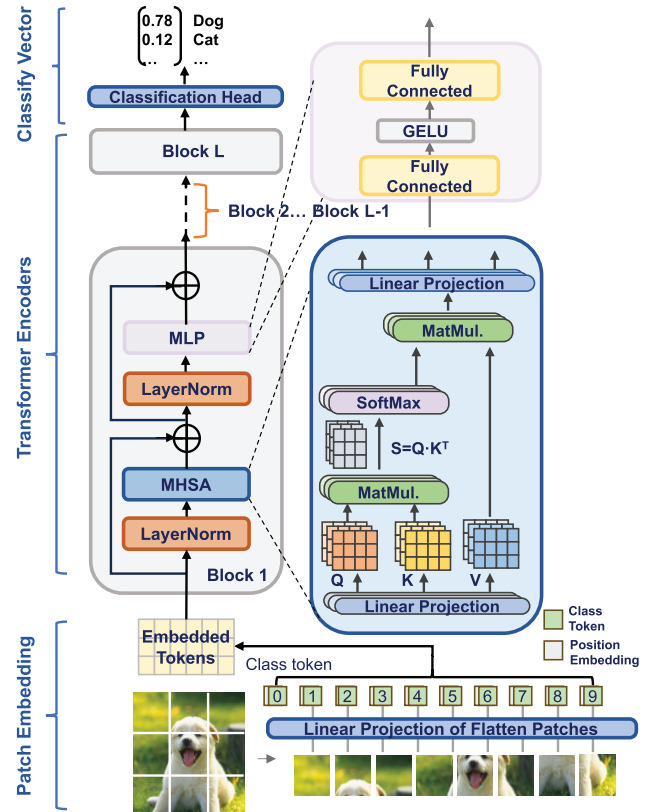


Fig. 3. The overall architecture and illustration of ViTs [4]. ViTs have numbers of repeated transformer encoder blocks with MHSA to capture the global information and MLP to feed-forward this perception. The input pictures will be slipped and embedded before being put into these blocks, and after numbers of operations, the classification heads will give out the probability scores.

equal to skip operations without any time consumption, which can leverage the characteristics of ViT weights and optimize

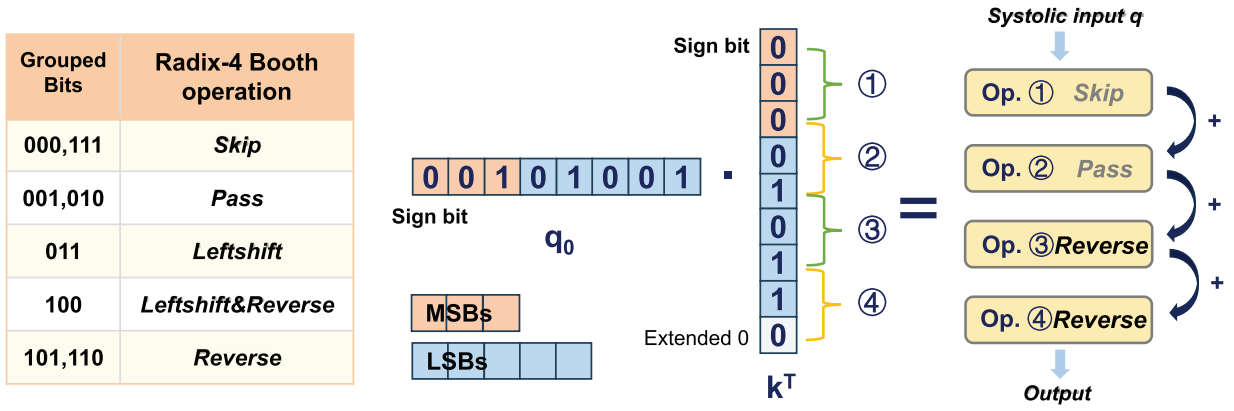


Fig. 4. Radix-4 Booth multiplication lookup table and instance. Booth multiplication divides the 8-bit extended multiplicand into four groups, transforming the multiplication of two numbers into a hardware-friendly combination of four logical operations on the multiplier. The specific operations are detailed in the Booth multiplication lookup table. Notably, the Booth operations corresponding to 000s and 111s do not require any computational time and can be skipped, which aligns well with the high density of 111s or 000s in ViTs. For example, in the case of multiplying 41 by 22, where the MSBs of 22 is 000s and the number of operations can be reduced by a quarter. The entire calculation process can be described as follows: 22 is encoded using Booth encoding for one skip, one pass, and two reverse operations. After logically shifting, the partial results of the pass and reverse operations will be summed to obtain the final result.

the end-to-end latency. (2) Using Booth-friendly LSB scaling algorithm to enlarge the proportion of continuous same 3-bits by scaling the 4th to 6th LSBs to 1s or 0s instead of simply setting them into zeros, which can not only minimize precision degradation but also maximize the benefits of Booth-Serial Skipping algorithm. (3) Pruning attention scores according to thresholds to further optimize the model's performance. The overall algorithm architecture is described in Alg. 1.

A. Booth-Serial Skipping Algorithm

As shown in Fig. 2, the same MSBs weights occupy 91.03% of the entire 8-bit quantized ViT models. In order to leverage the identical MSBs, we propose the Booth-Serial Skipping algorithm, which is depicted in Fig. 4: Before performing matrix multiplication, Q & K matrices are quantized into 8 bits, with K arranged in a serial format from MSBs to LSBs after zero extension. Served as a Booth multiplier, the zero-extended K is grouped and encoded into four operations according to the Radix-4 Booth multiple lookup table in Fig. 4. Each q in the Query matrix executes the operations step by step according to the encoded k , followed by shifting and accumulation to generate the final output. Based on the Booth operation lookup table, sequences of 111s and 000s in the Booth multiplier are encoded as skip operations, which means that both the memory access and computations of q are bypassed.

This skipping mechanism, derived from consecutive identical 1s and 0s, is termed Booth-Serial Skipping, with the proportion of skip operations among all operations defined as the Booth-skip rate. Given the strong correlation between the high Booth-skip rate in ViT weights and the Booth-Serial Skipping operation, the proposed algorithm leverages the potential of the naturally high density of identical MSBs in ViTs, replacing the energy-consuming multiplications to hardware-friendly operations. Besides that, the Booth-skip rate remains consistently high across both attention mechanisms and other linear computations. Consequently, the Booth-Serial Skipping Algorithm optimizes not only attention layers but also the end-to-end processing latency. To leverage the benefit,

we co-designed the Booth-Serial Processing Element (BSPE) which can perform Booth-Serial skipping operations in a systolic array architecture. The detailed design of BSPE is presented in Section IV.

B. Attention LSB Scaling

1) *Booth-Friendly Scaling*: Although Booth-Serial Early Skipping effectively leverages the high prevalence of identical MSB characteristics in quantized ViT parameters (e.g., exceeding 90% in DeiT base), the overall Booth-skip rate for 8-bit weights is not that competitive. To enhance the Booth-skip rate and maximize the benefit of the proposed Booth-Serial Skipping Algorithm with minimal accuracy degradation, we propose to modify the 4th to 6th bits of the Booth multiplier k compared to a predefined threshold. In this work, the threshold is set to 0, considering that $|K|_{\max}$ can be achieved when k is positive the 4_{th} to 6_{th} bits of the LSBs are set to 1s; otherwise, they are set to 0s. Since the exponentiation operation is required by SoftMax, only sufficiently large multiplicands contribute significantly. Therefore, we introduce a thresholding strategy: If k is bigger than 0, its 4th to 6th bits are set to 1s to maximize the operator, otherwise, they are set to 0s. Through scaling the 4th to 6th LSBs, we increase the Booth-skip rate from 36.5% to 56.4%, further reducing the latency by $1.5\times$ with little impact on accuracy.

2) *Attention Score Early Termination*: In order to optimize computational efficiency and reduce the associated data transfer overhead, we propose an attention score early termination algorithm inspired by [42] and [43]. During the finetune strategy we record the 10th and 90th percentiles of the values of $Q \times K^T$ in each layer as candidate thresholds and apply them as an optional threshold to selectively prune the attention scores during computation. Specifically, when the result of the $Q \times K^T$ falls below the threshold, its contribution to the SoftMax computation is negligible and we set the result to zero. Conversely, when the $Q \times K^T$ meets or exceeds the threshold, the corresponding value is retained for subsequent SoftMax processing due to its significant impact on

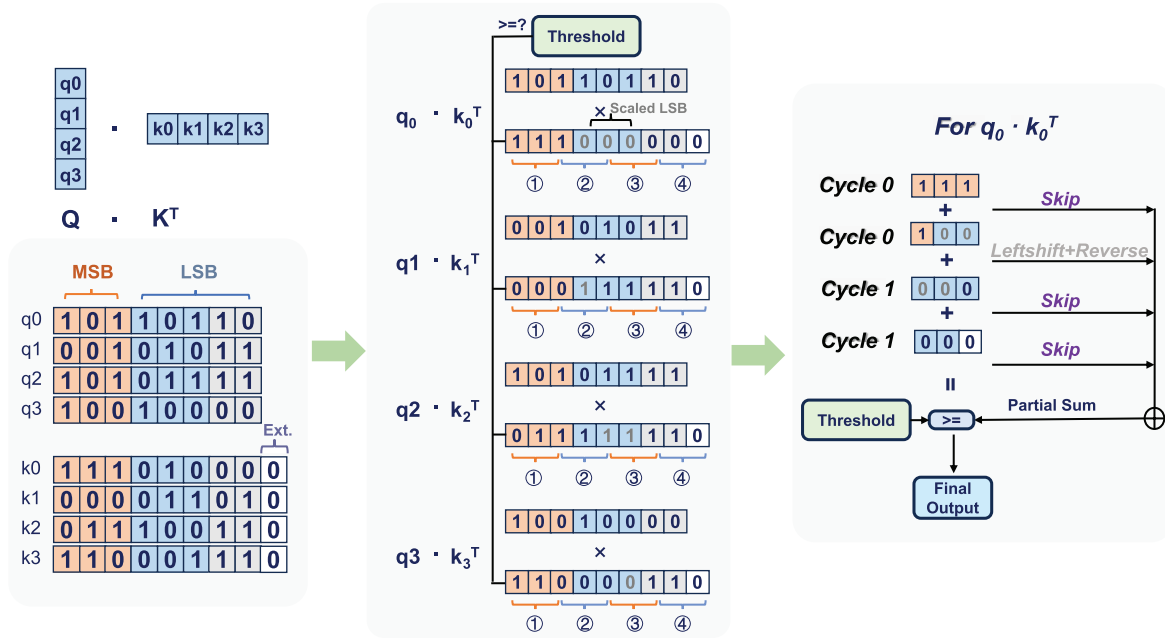


Fig. 5. The overall proposed BoostViT Algorithm. First, each k is zero-extended according to the Radix-4 Booth algorithm. To increase the Booth-skip rate while minimizing its impact on the approximation accuracy of the multiplication result, k is compared with a predefined threshold. If k exceeds the threshold, the 4th to 6th bits of the LSBs are set to 1s; otherwise, they are set to 0s. Booth multiplication is then performed following Booth-friendly LSB scaling. The resulting output is subsequently compared with the threshold: if the output exceeds the threshold, it is retained; otherwise, it is set to 0. Demonstrations show that after Booth-friendly LSB scaling, the required operation steps for k_0 are reduced from 2 to 1, thereby significantly improving the Booth-skip rate and reducing computational time.

the final attention score. This approach significantly reduces computational complexity by pruning lower-impact elements in SoftMax operation, thereby optimizing efficiency without compromising overall model accuracy.

V. PROPOSED BOOSTViT ACCELERATOR

A. Architecture Overview

The overview of BoostViT is shown in Fig. 6. Overall, BoostViT consists of a 16×16 PE array, two weight matrix buffers, an output buffer, an SRAM, an SM. unit, a nonlinear activation function unit, and a control unit. The PE array, which is responsible for all linear transformation computations, consists of Booth Skipping Processing Elements (BSPEs). These BSPEs support Booth multiplication within a systolic array and employ Booth-Serial Skipping to achieve end-to-end speedups. To enhance efficient throughput and PE utilization, each BSPE pipeline connects four Booth units, which can be dynamically reconfigured according to the Booth encoding requirements. Additionally, each BSPE is equipped with a small FIFO buffer to synchronize the control logic and support systolic operations, as processing times may vary between 0 and 4 cycles.

Compared to other accelerators, the primary innovation of BoostViT lies in eliminating the additional processing and storage overhead associated with indexing and masking operations, which are typically required to partition a matrix into sparse and dense regions. Moreover, BoostViT mitigates the unbalanced workloads caused by SpMM and SDDMM engines. Since the Booth-Serial Skipping Algorithm operates

at the bit level, all multipliers corresponding to a given multiplicand undergo the same operations based on the Booth code of the multiplicand, we employ a K-stationary systolic array to fully exploit the inherent data characteristics to achieve further acceleration.

B. BoostViT Accelerator's Micro-Architecture

1) *Internal Structure of the BSPE:* As shown in Fig. 6, each PE consists of a Booth encoder, four depth-reconfigurable pipelined Booth units that perform Booth shift operations (the pipeline is reconfigurable based on the Booth encoder, and if the k is 0, it behaves as a wire connecting its adjacent PEs), a FIFO responsible for handling unstructured latency for systolic array.

a) *Booth Encoder:* Before computation, each PE receives the input k . This k is segmented and encoded into four parts according to the Booth lookup table, which uses combinational logic to configure the corresponding multiplication logic within the pipeline accordingly.

b) *Booth Unit:* Each Booth Unit incorporates a gated clock and a Booth Multiplier, which can be reconfigured according to the Booth Encoder to perform different computational operations. As described in Fig. 7 (a), when the corresponding Booth code is either 111s or 000s, the Booth Unit performs Booth-Serial Skipping, allowing the unit to be configured solely as a bypass wire without consuming any clock cycles. This optimization effectively reduces the observable pipeline depth by one stage while maintaining the logical functionality of a four-stage pipeline, thereby ensuring that four multiplication operations are processed simultaneously.

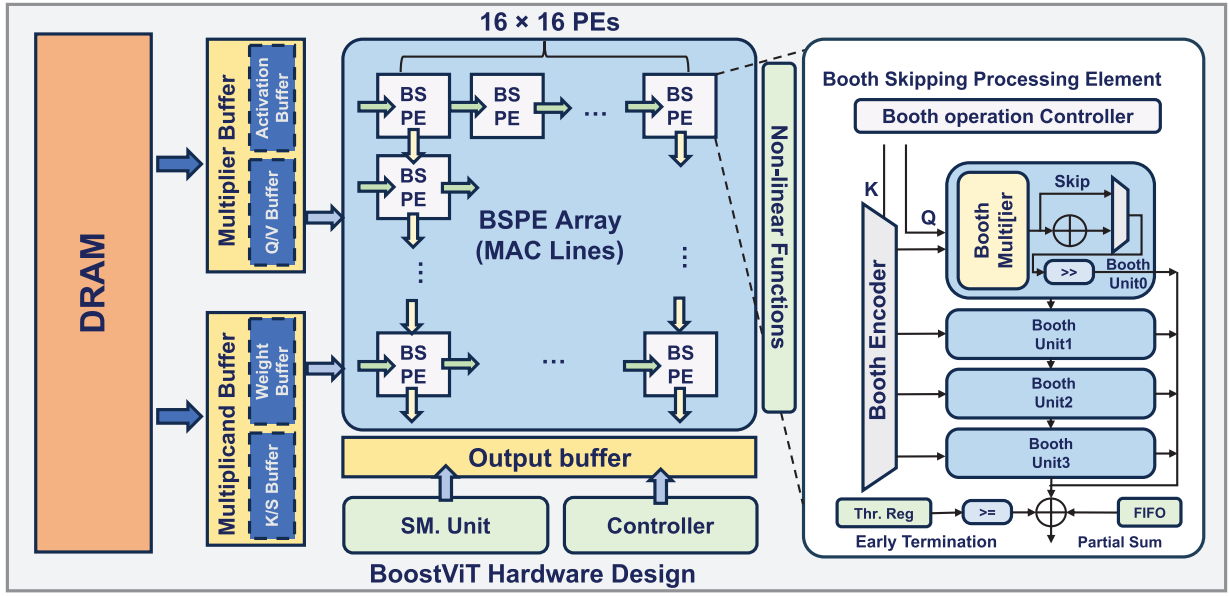


Fig. 6. Illustration of the micro-architecture of our BoostViT accelerator. The overall controller is responsible for managing the overall dataflow. The Q/V and K/S buffers are used to store the multipliers and multiplicands matrices, which will be sent to the BSPE array. The BSPE array serves as the GEMM computation core, consisting of 16×16 Booth-Serial processing elements that can perform Booth-skip in a K-stationary systolic array. The SM. Unit performs exponential, accumulation, and reciprocal operations, which serve as the fundamental computations for both SoftMax and LayerNorm. The inner architecture depicts the detailed BSPE design. The Booth encoder configures the four pipelined Booth Units according to the lookup table of the Booth-friendly LSB scaled and 0-extended k , and if the value is 000s or 111s the corresponding Booth Unit is configured into a wire. Each Booth Unit processes the received Booth multiplier value and accumulates it with the partial sums from previous computations. The final output will be compared to the threshold and stored in the Booth-FIFO, waiting for the handshake signals and systolic setup. For the Q/K/V generation and MLP layers, since the weights in these other linear mapping layers—like the K matrix in attention—can also be Booth-encoded to produce significant Booth-skip rates, BoostViT can also enjoy an end-to-end speed up via reusing the BSPE Array.

c) *Booth FIFO*: As described in Fig. 7 (b), the unstructured distribution of 111s or 000s patterns in weights presents a timing challenge: the pipeline depth can vary from 0 to 4 cycles across different k . Consequently, an input q might produce an output from certain PEs at cycle 1, whereas others may not generate an output until cycle 5. This misalignment conflicts with the strict timing constraints of the systolic array used for general matrix multiplication. To ensure proper synchronization, we introduce a FIFO buffer with a depth corresponding to the maximum timing difference between adjacent PEs, along with handshake signals between the left and right neighboring PEs. Let the pipeline depth of the $(K - 1)_{th}$ PE after configuration be denoted as a_1 . At time t_0 , it receives the first row of input q_{01} , and will output the first row s_{01} at time $t_0 + 2a_1$. For the PE located to the left, the pipeline depth of the $(K - 1)_{th}$ PE is denoted as a_0 . Due to the pulsation sequence of the systolic array, it receives the first row of input q_{00} at time $t_0 - 1$, and will output the first row s_{00} at time $t_0 - 1 + 2a_0$. Similarly, the $(K + 1)_{th}$ PE will output the first row s_{02} at time $t_0 + 1 + 2a_2$. Thus, the FIFO depth must meet the following situation:

$$\max\{2|a_1 - a_0| + 1, 2|a_2 - a_1| + 1\} \quad (2)$$

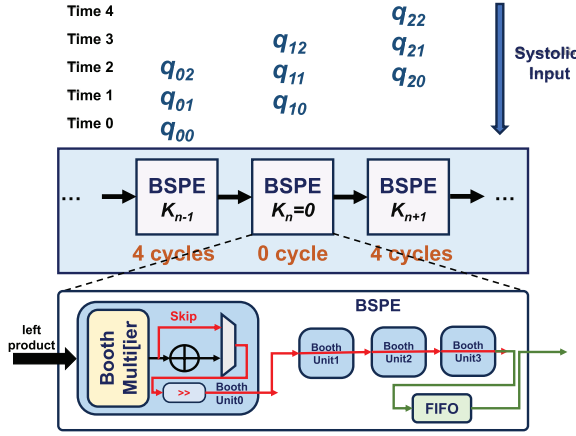
Fig. 7(b) simulates the worst case for the Radix-4 Booth algorithm, when the center PE is configured as a wire, while its adjacent PEs execute full computations. The maximum setup time in this case is 9 cycles and we set the depth of the FIFO to 9 to ensure the systolic array sequence requirements are met.

2) *BSPE Array's Workload and Nonlinear Function Approximation*: The overall workload of BoostViT's hardware

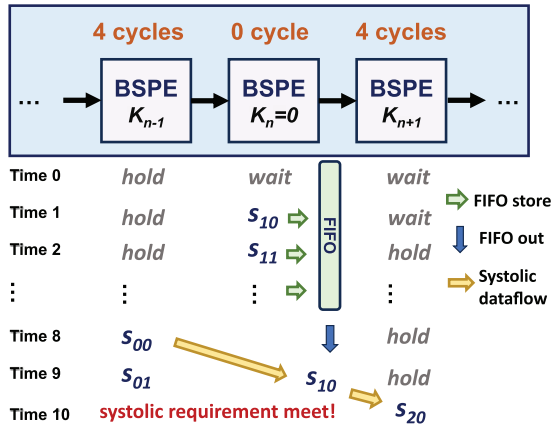
can be described as follows: **For the core attention part**, before the q is pulsed and transmitted downward, each BSPE stores its corresponding k and reconfigures its four Booth units to perform the required operations. After completing the operation (including Booth skipping), the pipelined operation results are stored in a FIFO and forwarded to the adjacent BSPE on the right. This BSPE accumulates its computed output with the data received from the preceding PE before passing the cumulative result to the next PE. Ultimately, the final computed result is stored in the output buffer. Fig. 8 shows the visualized operation requirements in different Booth-skip rate layers. When the Booth-skip rate is relatively high (e.g. 57% in the attention layer after Booth-friendly LSB scaling), most Booth Units are configured as bypass wires, leading to significant speedup and improved energy efficiency. **For the linear projection** and feed-forward network (FNN) layers, as shown in Fig. 2, they also exhibit a considerable Booth-skip rate. Our accelerator architecture can also be further reused for the Q/K/V generation and MLP layers similarly employ multiplier-stationary the BSPE array to perform Booth-skipping of the linear transform, thereby achieving greater end-to-end inference benefits rather than limiting optimization solely to the attention computation. For the GLUE [44] function and e^x used in softmax, we apply the following approximations:

$$\text{GELU}(x) \approx 0.5 \cdot x \left(1 + ((x \cdot 0.7978) + (x^2 \cdot 0.0447)) \right) \quad (3)$$

$$e^x \approx 1 + x + x^2 \times 0.5 + x^3 \times 0.1667 + x^4 \times 0.0417 + x^5 \times 0.0083 \quad (4)$$



(a) Inner configure of the BSPE performing Booth Skipping



(b) Systolic mapping requirement for FIFO

Fig. 7. In operation, a PE first acquires a key (k) during initialization, after which the encoder configures the corresponding multiplexer operations based on the Booth lookup table. Once a query (q) is received, the input is processed through the pipeline and stored in the FIFO. The PE then issues a read signal to its left neighbor and waits for a corresponding signal from its right neighbor. When the left-hand output is received and the right-hand read signal is detected, the systolic array aligns the cumulative results, thereby completing the Booth-based skip multiplication and coupling with the systolic array.

VI. EXPERIMENTS

A. Experiment Setting

1) *Models, Datasets, and Training Settings*: Our experiments were conducted on the ImageNet-1K [45] dataset using several widely recognized ViT models [29], [33], [37]. Initially, these models were quantized to 8-bit integers. Subsequently, we applied our proposed Booth Serial Skipping algorithm for optimization, followed by fine-tuning using the approach outlined in [29]. We configured variable thresholds for each model, achieving spasticities of $Q \times K^T$ ranging from 10% to 90%.

2) *Baselines and Evaluation Metrics*: **Baselines**: To benchmark BoostViT against other ViT accelerators on ASIC, we compared it with six baselines, including general-purpose platforms: CPU(Intel Xeon Gold 6230R), EdgeGPU(NVIDIA Jetson Xavier NX), and GPU(NVIDIA 2080Ti), as well as state-of-art ViT accelerators, Sanger [12], ViTCoD [14] and ViT-slice [19]. **Metrics**: We evaluated all platforms in

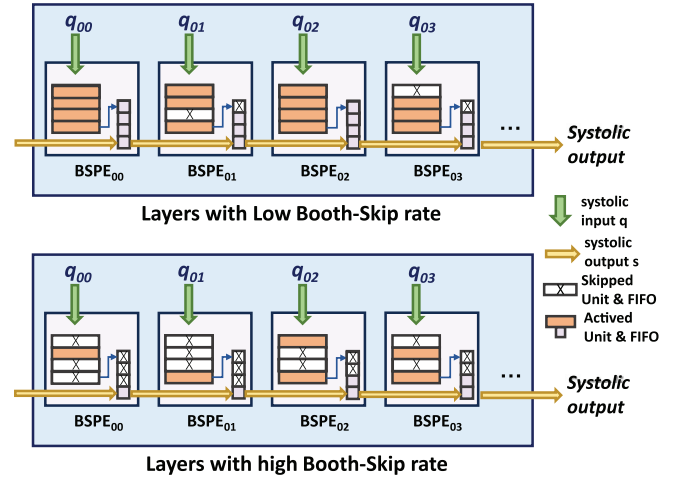


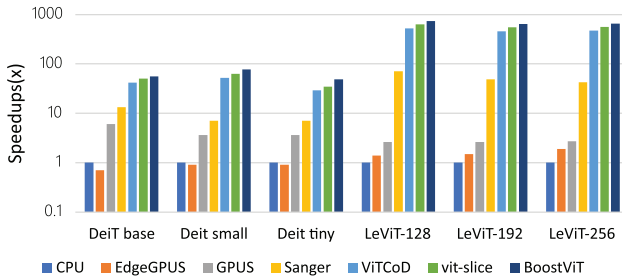
Fig. 8. The visualized non-wire Booth units in different Booth-skip rate layers. Although different BSPEs may be configured with varying pipeline depths for different k values, we employ a FIFO buffer to store outputs that may be temporally misaligned. When the number of skip operations differs across BSPEs, the earliest outputs are held in the FIFO until the adjacent BSPE with the greatest latency issues its handshake signal. Consequently, after the pulsed pipeline is established, the BSPE array operates in a stable, balanced state, thereby avoiding workload imbalance.

terms of speedup and efficiency throughput. To ensure a fair performance comparison, we scale all the accelerators' hardware resources for similar configurations following the methodology of DOTA [46] and ViTCoD [14]. For other accelerator benchmarks, we compare the **Efficient Throughput** according to Sanger [12], defining the peak computational performance as the number of multiplications performed concurrently by the PE array. For BoostViT, this value is given by $\text{Throughput} \times \text{Computation saving} \times \text{Operations}$. Due to the deep pipelined design within each PE, BoostViT significantly enhances data throughput. It is important to note that while the Booth skipping operation simplifies the logic—wherein n Booth Units effectively behave as a direct connection wire (yielding a physical pipeline depth of $4 - n$)—the logical computation still involves four multiplications, meaning that the effective pipeline depth remains 4. To benchmark BoostViT against other ViT accelerators on FPGA, we compared it with state-of-the-art works, namely BSViT [36], HeatViT [40], [47] and [48].

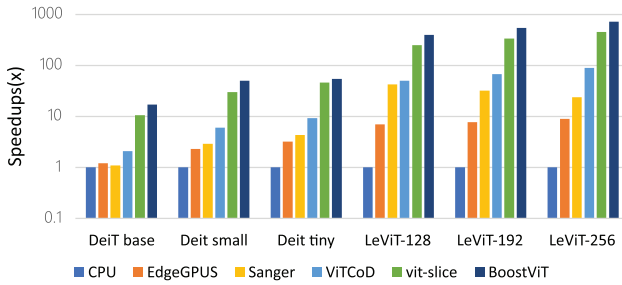
3) *ASIC & FPGA Implementation*: **Characteristics**: For the ASIC implementation, BoostViT is designed with a total area of 4.06 mm^2 and equipped with a 300KB SRAM and 256 PEs (each PE has 4 Booth Unit) under the DDR4 memory bandwidth of 76.8GB/s. The power consumption of the hardware architectures is 785.039 mW at a core frequency of 500MHz, 45nm CMOS technology, achieving an energy efficiency of 2.608 TOPs/w. Mentioned that the FIFO in each BSPE is extremely small, they only contain an energy consumption of 76.8 mw and an area of 0.438 mm^2 , which is negligible for the performance increase. The SRAM includes (1) Q/K/S/V memory occupies 128KB, (2) output buffer occupies 64KB; and (3) weight/activation memory occupies 108 KB. **Evaluation**: We developed a cycle-accurate simulator, for which we considered the Booth sparsity via simulating functions and the memory

TABLE I
COMPARISON OF VARIOUS ACCELERATORS FOR VISION AND NLP TRANSFORMERS

Accelerator	A ³	SpAtten	Sanger	ViTCoD	HeatViT	ViT-slice	BoostViT (This work)
Application Field	NLP Transformer	NLP Transformer	NLP Transformer	Vision Transformer	Vision Transformer	Vision Transformer	Vision Transformer
Workloads	Sparse Attention: SDDMM; SpMM	Sparse Attention: SDDMM; SpMM	Sparse Attention: SDDMM; SpMM	Sparse Attention: SDDMM; SpMM	Pruning	MSB Approximation	Booth-Serial Skipping Algorithm
Dataflow	Greedy Search	Top-k Selection	S-stationary	K-stationary	GEMM	Bit-level Skip	Booth-skip K-stationary
Acceleration Strategy	Multiplier selected Unstructured	Multiplier selected Unstructured	Fine-grained Structured	Threshold Binarization	Token Selected Structured	Threshold Binarization	Threshold LSB Scaling
Sparsity	Medium	Low	Medium-High	High	High	High	High



(a) Attention Speedups (90% Sparsity)



(b) End-to-end ViTs Speedups

Fig. 9. Attention speedups and end-to-end speedups comparison between BoostViTs and other baselines.

access cost. Specifically, we synthesized our RTL design with a commercial 45nm CMOS technology using Synopsys tools (Design Compiler [49] for gate level netlist and the Memory Compilers from the foundry). For the FPGA implementation, we implemented BoostViT on the Xilinx ZCU102 platform with Zynq UltraScale+ MPSoC using Vivado platform.

B. Overall Performance Comparison

1) *Comparison With ASIC-Based State-of-the-Art Accelerators*: Fig. 9 shows the attention and end-to-end speedup comparisons of BoostViT and six baselines. Note that the sparsity is adjusted to maintain accuracy within 1.5% drops, and we implement and simulate both of them on ViTs with similar hardware configurations for fair comparisons. On average, BoostViT achieves 60.3 $\times$, 66.9 $\times$, 16.7 $\times$ attention speedup, and 50.3 $\times$, 21.9 $\times$, 17.37 $\times$ end-to-end speedup over CPU (Intel Xeon Gold 6152), EdgeGPU (NVIDIA Jetson Xavier

TABLE II
COMPARISON ON END-TO-END PERFORMANCE WITH STATE-OF-THE-ART VISION TRANSFORMER ACCELERATORS

Metric	JSSC'24 [50]	ViTCoD	BSViT	This work
Model	LLMs/ViTs ²	DeiT-T	DeiT-T	DeiT-T
Voltage (V)	1.0	1.0	1.08	1.0
Performance (TOPS)	0.17-6.53@INT8	0.612@INT8	0.441@INT8/4	2.048 ³ @INT8
Energy Efficiency (TOPS/W)	4.75 ²	1.889	6.973 ¹	6.760 ¹

¹ Our design and BSViT are scaled to 28nm technology using the scaling equations used in [36] and [51]: $f_1/f_2 = s$, $Area_2/Area_1 = s^2$, $p_2/p_1 = s$, where s is technology scaling factor.

² Mentioned the reported performance includes both language and vision transformer modules since JSSC'24 [50] only reports the average energy efficiency over 12 typical models and tasks (BERT-B/GPT-2/LLaMA-7B/ViT-B).

³ We calculate the Efficient Throughput under the Booth-skipping rate of 50%.

NX), GPU (TITAN Xp) on DeiT. Additionally, we compare the attention and end-to-end speedup of BoostViT with SOTA Transformer accelerators, such as Sanger [12], ViTCoD [14], and ViT-slice [19]. As a result, BoostViT achieves an average **attention speedup** of 8.49 $\times$, 1.49 $\times$, and 1.24 $\times$ over Sanger, ViTCoD, and ViT-slice, respectively. Furthermore, BoostViT outperforms ViTCoD and ViT-slice on **end-to-end** models with 7.47 $\times$ and 1.49 $\times$ speedup, respectively. While ViTCoD concentrates solely on optimizing attention and ViT-slice only takes advantage of the MSB characteristics, BoostViT achieves a higher improvement in end-to-end speedup compared to the attention speedup by taking full advantage of our Booth-Serial Skipping algorithm. Table II presents the energy efficiency comparison between ViTCoD, BSViT, and JSSC'24 [50]. Compared to ViTCoD, BoostViT not only optimizes attention computation but also linear transformation operations, resulting in significant improvements in energy efficiency. For JSSC'24 [50], the authors propose the Heterogeneous-Dataflow PE Array and other contributions that accelerate both attention computations and linear transformations. However, because [50] is designed as a general-purpose transformer accelerator rather than being specifically optimized for ViT models, its average energy efficiency falls short of that achieved by BoostViT after scaling to the same technology node. Although BoostViT shows slightly lower energy efficiency compared to BSViT, which is implemented using 65nm technology, it achieves nearly 5 $\times$ higher performance. Considering that process scaling may introduce certain estimation inaccuracies, we believe our design demonstrates similarly competitive energy efficiency.

TABLE III
FPGA ACCELERATORS PERFORMANCE COMPARISON (THROUGHPUT AND RESOURCE UTILIZATION)

Accelerator	FPGA Platform	Data Type	Frequency (MHz)	kLUT	DSP	BRAM	Efficient Throughput (GOPs)
BoostViT (This work)	ZCU 102	int8	300	211.6	198	539.5	853.3
TCAS-I'24 BSViT [36]	Alveo U280	int8/4	300	253	224	760	532.9
TCAS-I'23 [47]	ZCU 102	int8	300	144.5	1268	648	616.1
HPCA'23 HeatViT [40]	ZCU 102	int8	300	137.6	1968	355.5	203

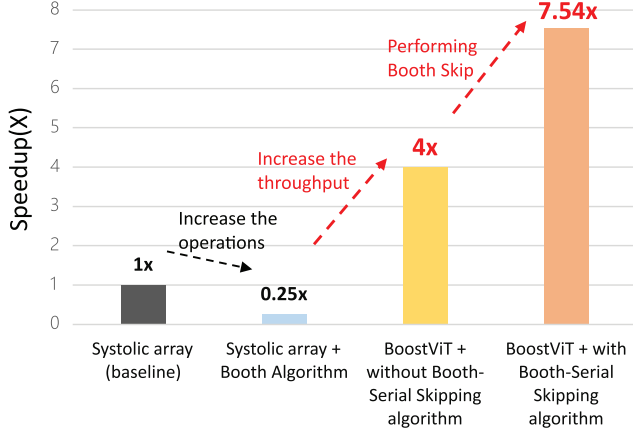


Fig. 10. Breakdown of average speedup achieved by BoostViT on DeiT-Tiny/Small/Base models.

2) *Comparison With FPGA-Based State-of-the-Art Accelerators*: We implemented an FPGA-based design with a 16×16 PE array at 300Mhz and compared it with related SOTA FPGA-based works. The detailed comparison is presented in Table III. Our Booth-Serial Skipping Algorithm leverages the bit-level sparsity of ViTs and the reconfigurable pipelined BSPEs contribute to the highly performance, reaching up to an efficient throughput of 853.3 GOP/s. BSViT [36] tries to optimize the dense and sparse engine at bit-level, but its fixed 4bit-4bit grouping strategy results in lower throughput and higher resource utilization. In [47], their designs heavily occupy DSP resources for competitive throughput. For HeatViT [40], they only focus on pruning the tokens without a deeply co-designed hardware framework, resulting in relatively lower throughput.

C. Performance Analysis

1) *Evaluation of the BoostViT Accelerator*: For the **attention part**, ViTCoD's unbalanced SpMM and SDDMM workload makes the SDDMM engine the bottleneck of its efficient throughput (e.g., the SDDMM engine needs 7 times cycles more than the SpMM engine according to their simulators). Similarly, ViT-slice only takes usages of the 3-bits out of the full 8-bits representation, limiting its efficiency. In contrast, our Booth-Serial Skipping algorithm not only leverages the MSBs but also significantly enhances the Booth-skip rate in attention operations. Specifically, the Booth-skip rate is

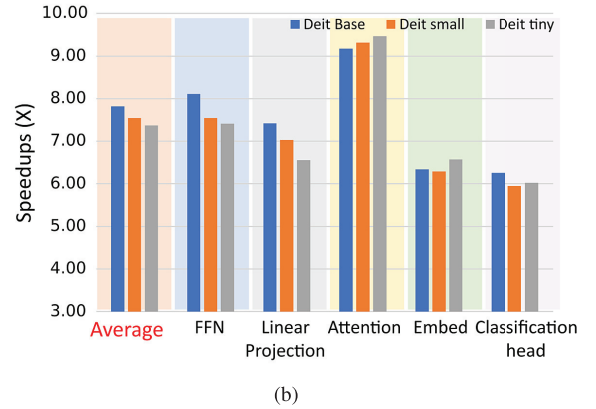
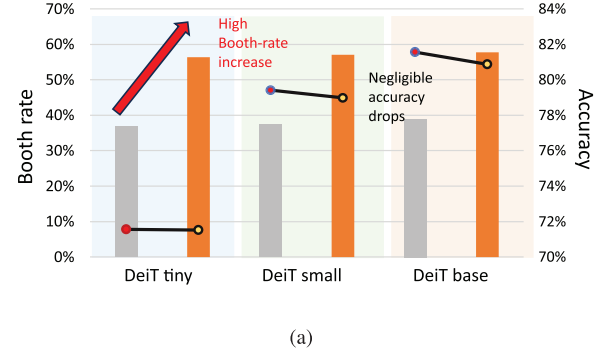


Fig. 11. Impact of the Attention LSB scaling on accuracy and skipping rate and the speedup of BoostViT algorithm in each layer. (a) shows the trade-off between accuracy and Booth-skip rate, (b) shows the speedup contributions from BoostViT in each layer.

magnified from 38.93% to 57.74% thanks to our attention Booth-friendly LSB scaling algorithm, increasing the speedups from $6.7\times$ to $9.3\times$. **From the end-to-end perspective**, the natural proportion of 111s and 000s existing in the FFN layer is as high as 45%, which allows our Booth-Serial Skipping Algorithm to achieve a $1.82\times$ speedup without additional overhead.

2) *Ablation Studies on BoostViT Hardware*: Fig. 10 shows the performance breakdown of BoostViT in the DeiT-Tiny/Small/Base models, using a GEMM systolic array as the baseline. After performing the Booth Algorithm, 1 cycle multiply is divided into 4 cycles, and the operation increase results in a reduction of the speedup to $0.25\times$ compared to the baseline. To fully exploit the benefits of both dense identical MSBs and the Booth Algorithm, BoostViT integrates a pipelined Booth Unit design while ensuring the

proper functionality of the systolic array. As a result, the BoostViT (without Booth-Serial Skipping algorithm) achieves $4\times$ speedup. In addition, the Booth-Serial Skipping algorithm co-designed BSPE can fully leverage the identical 3-bits characteristics in ViT's weight and activation—which account for 46% of linear operations and 60% of attention calculations. As a result, BoostViT (with Booth-Serial Skipping algorithm) achieves a substantial $7.54\times$ speedup compared to the baseline.

3) *Booth-Serial Skipping Algorithm Analyze*: The original Booth-skip rate of the attention part is nearly 37%, not as high as the identical MSBs proportion. In order to magnify the natural Booth-skip rate and minimize the accuracy drops, our Booth-serial skipping algorithm shifted the 4th to 6th bits of key vectors to either 000s or 111s. These bits contribute less to the final output compared to the MSBs, yet their adjustment effectively increases the Booth-skip rate in the attention process to approximately 57%, thereby further accelerating attention computation. Fig. 11 (a) shows the negligible accuracy drops and increasing Booth-skip rate benefits from our proposed algorithm, demonstrating that our algorithm can reduce exceptional computations with zero harm on accuracy. Fig. 11 (b) demonstrates that our BoostViT not only has significant optimizations of attention but also efficiently speeds up all other kinds of layers. These results explain why BoostViT achieves a much higher speedup speedups in both the end-to-end and attention parts compared to ViTCoD and ViT-slice.

VII. CONCLUSION

In this paper, we present BoostViT, one software-hardware co-designed framework to leverage the bit character in ViTs. At the algorithm level, we proposed a learnable threshold to early terminate and scale the LSB to boost the opportunity to perform Booth-Serial Skipping. In the hardware aspect, BoostViT employs an orchestrated BSPE for Booth logics and systolic supputation. BSPE owns four pipelined Booth Units with clock-gating design, which are reconfigurable for different multipliers, enhancing overall throughput. This BSPE array is not just applied to perform the attention module, but also other linear modules, including FFN, linear projection, and linear transformation, increasing the end-to-end speedups. BoostViT achieves an end-to-end speedup of $50.3\times$, $21.9\times$, $17.37\times$, $7.47\times$ and $1.49\times$ over CPU, EdgeGPU, GPU, ViTCoD, and ViT-slice on DeiT, respectively.

REFERENCES

- [1] D. Bahdanau, K. Cho, and Y. Bengio, "Neural machine translation by jointly learning to align and translate," 2014, *arXiv:1409.0473*.
- [2] A. P. Parikh, O. Täckström, D. Das, and J. Uszkoreit, "A decomposable attention model for natural language inference," 2016, *arXiv:1606.01933*.
- [3] A. Vaswani et al., "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 30, Jun. 2017, pp. 5998–6008.
- [4] A. Dosovitskiy et al., "An image is worth 16×16 words: Transformers for image recognition at scale," 2020, *arXiv:2010.11929*.
- [5] M. Chen et al., "Generative pretraining from pixels," in *Proc. Int. Conf. Mach. Learn.*, 2020, pp. 1691–1703.
- [6] N. Carion, F. Massa, G. Synnaeve, N. Usunier, A. Kirillov, and S. Zagoruyko, "End-to-end object detection with transformers," in *Proc. Eur. Conf. Comput. Vis.* Cham, Switzerland: Springer, 2020, pp. 213–229.
- [7] S. Zheng et al., "Rethinking semantic segmentation from a sequence-to-sequence perspective with transformers," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2021, pp. 6881–6890.
- [8] H. Chen et al., "Pre-trained image processing transformer," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2021, pp. 12299–12310.
- [9] K. Han et al., "A survey on vision transformer," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 45, no. 1, pp. 87–110, Jan. 2022.
- [10] M. Treviso, A. Góis, P. Fernandes, E. Fonseca, and A. F. T. Martins, "Predicting attention sparsity in transformers," 2021, *arXiv:2109.12188*.
- [11] K. Kim et al., "Rethinking the self-attention in vision transformers," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. Workshops (CVPRW)*, Jun. 2021, pp. 3065–3069.
- [12] L. Lu et al., "Sanger: A co-design framework for enabling sparse attention using reconfigurable architecture," in *Proc. 54th Annu. IEEE/ACM Int. Symp. Microarchitecture*, Oct. 2021, pp. 977–991.
- [13] H. Wang, Z. Zhang, and S. Han, "SpAtten: Efficient sparse attention architecture with cascade token and head pruning," in *Proc. IEEE Int. Symp. High-Perform. Comput. Archit. (HPCA)*, Feb. 2021, pp. 97–110.
- [14] H. You et al., "ViTCoD: Vision transformer acceleration via dedicated algorithm and accelerator co-design," in *Proc. IEEE Int. Symp. High-Perform. Comput. Archit.*, Sep. 2023, pp. 273–286.
- [15] T. Chen, Y. Cheng, Z. Gan, L. Yuan, L. Zhang, and Z. Wang, "Chasing sparsity in vision transformers: An end-to-end exploration," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 34, Jan. 2021, pp. 19974–19988.
- [16] Y. Rao, W. Zhao, B. Liu, J. Lu, J. Zhou, and C. J. Hsieh, "DynamicViT: Efficient vision transformers with dynamic token sparsification," in *Proc. Adv. Neural Inf. Process. Syst. (NIPS)*, 2021, pp. 13937–13949.
- [17] Y. Tang et al., "Patch slimming for efficient vision transformers," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2022, pp. 12165–12174.
- [18] Y. Xu et al., "Evo-ViT: Slow-fast token evolution for dynamic vision transformer," in *Proc. AAAI Conf. Artif. Intell.*, vol. 36, no. 3, 2022, pp. 2964–2972.
- [19] D. Shin, I. Choi, and J.-S. Yang, "ViT-slice: End-to-end vision transformer accelerator with bit-slice algorithm," in *Proc. 61st ACM/IEEE Design Autom. Conf.*, Jun. 2024, pp. 1–6.
- [20] M. Fayyaz et al., "Adaptive token sampling for efficient vision transformers," in *Proc. Eur. Conf. Comput. Vis.* Cham, Switzerland: Springer, 2022, pp. 396–414.
- [21] N. Srivastava, H. Jin, J. Liu, D. Albonesi, and Z. Zhang, "MatRaptor: A sparse-sparse matrix multiplication accelerator based on row-wise product," in *Proc. 53rd Annu. IEEE/ACM Int. Symp. Microarchitecture (MICRO)*, Jun. 2020, pp. 766–780.
- [22] C. Zheng et al., "SAViT: Structure-aware vision transformer pruning via collaborative optimization," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 35, 2022, pp. 9010–9023.
- [23] K. Hegde et al., "Extensor: An accelerator for sparse tensor algebra," in *Proc. 52nd Annu. IEEE/ACM Int. Symp. Microarchitecture (MICRO)*, Sep. 2019, pp. 319–333.
- [24] S. Pal et al., "OuterSPACE: An outer product based sparse matrix multiplication accelerator," in *Proc. IEEE Int. Symp. High Perform. Comput. Archit. (HPCA)*, Aug. 2018, pp. 724–736.
- [25] B. Pan, R. Panda, Y. Jiang, Z. Wang, R. Feris, and A. Oliva, "IA-RED²: Interpretability-aware redundancy reduction for vision transformers," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 34, 2021, pp. 24898–24911.
- [26] I. Bello, B. Zoph, Q. Le, A. Vaswani, and J. Shlens, "Attention augmented convolutional networks," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2019, pp. 3286–3295.
- [27] B. Wu et al., "Visual transformers: Token-based image representation and processing for computer vision," 2020, *arXiv:2006.03677*.
- [28] S. Zhao et al., "A 0.99-to-4.38 uJ/class event-driven hybrid neural network processor for full-spectrum neural signal analyses," *IEEE Trans. Biomed. Circuits Syst.*, vol. 17, no. 3, pp. 598–609, Jun. 2023.
- [29] H. Touvron, M. Cord, M. Douze, F. Massa, A. Sablayrolles, and H. Jeéou, "Training data-efficient image transformers & distillation through attention," in *Proc. Int. Conf. Mach. Learn.*, Jul. 2021, pp. 10347–10357.
- [30] C.-F. R. Chen, Q. Fan, and R. Panda, "CrossViT: Cross-attention multi-scale vision transformer for image classification," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, Oct. 2021, pp. 357–366.
- [31] B. Heo, S. Yun, D. Han, S. Chun, J. Choe, and S. J. Oh, "Rethinking spatial dimensions of vision transformers," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2021, pp. 11936–11945.
- [32] W. Wang et al., "Pyramid vision transformer: A versatile backbone for dense prediction without convolutions," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2021, pp. 568–578.

- [33] Z. Liu et al., “Swin transformer: Hierarchical vision transformer using shifted windows,” in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2021, pp. 10012–10022.
- [34] H. Wu et al., “CvT: Introducing convolutions to vision transformers,” in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2021, pp. 22–31.
- [35] S. Mehta and M. Rastegari, “MobileViT: Light-weight, general-purpose, and mobile-friendly vision transformer,” 2021, *arXiv:2110.02178*.
- [36] G. Wang, S. Cai, W. Li, D. Lyu, and G. He, “BSViT: A bit-serial vision transformer accelerator exploiting dynamic patch and weight bit-group quantization,” *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 71, no. 9, pp. 4064–4077, Sep. 2024.
- [37] B. Graham et al., “LeViT: A vision transformer in ConvNet’s clothing for faster inference,” in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2021, pp. 12259–12269.
- [38] T. J. Ham et al., “A3: Accelerating attention mechanisms in neural networks with approximation,” in *Proc. IEEE Int. Symp. High Perform. Comput. Archit. (HPCA)*, May 2020, pp. 328–341.
- [39] S. Yu et al., “Unified visual transformer compression,” 2022, *arXiv:2203.08243*.
- [40] P. Dong et al., “HeatViT: Hardware-efficient adaptive token pruning for vision transformers,” in *Proc. IEEE Int. Symp. High-Perform. Comput. Archit. (HPCA)*, Feb. 2023, pp. 442–455.
- [41] C. Fang et al., “A 0.078 pJ/SOP unstructured sparsity-aware spiking attention/convolution processor with 3D compute array,” in *Proc. IEEE Custom Integr. Circuits Conf. (CICC)*, Apr. 2024, pp. 1–2.
- [42] Z. Li, S. Ghodrati, A. Yazdanbakhsh, H. Esmailzadeh, and M. Kang, “Accelerating attention through gradient-based learned run-time pruning,” in *Proc. 49th Annu. Int. Symp. Comput. Archit.*, Jun. 2022, pp. 902–915.
- [43] J. Park, M. Kang, Y. Han, Y.-G. Kim, J. Shin, and L.-S. Kim, “Token-picker: Accelerating attention in text generation with minimized memory transfer via probability estimation,” in *Proc. 61st ACM/IEEE Design Autom. Conf.*, Jun. 2024, pp. 1–6.
- [44] D. Hendrycks and K. Gimpel, “Gaussian error linear units (GELUs),” 2016, *arXiv:1606.08415*.
- [45] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, “ImageNet: A large-scale hierarchical image database,” in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, Jun. 2009, pp. 248–255.
- [46] Z. Qu, L. Liu, F. Tu, Z. Chen, Y. Ding, and Y. Xie, “DOTA: Detect and omit weak attentions for scalable transformer acceleration,” in *Proc. 27th ACM Int. Conf. Architectural Support Program. Lang. Operating Syst.*, Feb. 2022, pp. 14–26.
- [47] M. Huang, J. Luo, C. Ding, Z. Wei, S. Huang, and H. Yu, “An integer-only and group-vector systolic accelerator for efficiently mapping vision transformer on edge,” *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 70, no. 12, pp. 5289–5301, Dec. 2023.
- [48] E. Kwon, J. Yoon, and S. Kang, “Mobile transformer accelerator exploiting various line sparsity and tile-based dynamic quantization,” *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 43, no. 6, pp. 1808–1821, Jun. 2024.
- [49] (2016). *Synopsys Design Compiler*. [Online]. Available: <http://www.pages/default.aspx>
- [50] Y. Qin et al., “Ayaka: A versatile transformer accelerator with low-rank estimation and heterogeneous dataflow,” *IEEE J. Solid-State Circuits*, vol. 59, no. 10, pp. 3342–3356, Oct. 2024.
- [51] W. Li, A. Hu, N. Xu, and G. He, “A precision-scalable deep neural network accelerator with activation sparsity exploitation,” *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 43, no. 1, pp. 263–276, Jan. 2024.